

DOCUMENTATION TECHNIQUE

Serveur de ticketing GLPI sécurisé

Installation, sécurisation, exploitation et reprise du service

Auteur	Maël Jansen, Kilian Levasseur
Date	3 septembre 2026
Serveur	<code>ubuntuserver2204</code> – Ubuntu Server 22.04 LTS
Adresse locale	192.168.1.17 (<code>enp0s8</code>)
Applicatif	GLPI 10 – MariaDB 10.11 – NGINX, sous Docker Compose
Accès au service	HTTPS, uniquement depuis le tunnel VPN
Référentiel	Cahier de recettes du projet
Dépôt	systeme-de-ticketing-securise

Sommaire

Sommaire.....	2
1. Présentation du service.....	4
1.1 Objet et périmètre.....	4
1.2 Architecture.....	4
1.3 Inventaire de la plateforme.....	4
1.4 Composants applicatifs et persistance.....	5
1.5 Matrice des flux réseau.....	5
1.6 Arborescence des fichiers.....	6
1.7 Le principe retenu : la défense en profondeur.....	7
2. Procédure d'installation.....	8
2.1 Prérequis.....	8
2.2 Mise à jour du socle.....	8
2.3 Installation de Docker.....	8
2.4 Arborescence et secrets.....	9
2.5 Fichier de composition.....	9
2.6 Premier démarrage.....	10
2.7 Assistant d'installation de GLPI.....	11
2.8 Durcissement immédiat des comptes.....	12
2.9 Contrôles de fin d'installation.....	14
3. Procédure de sécurisation.....	15
3.1 Certificat TLS.....	15
3.2 Reverse proxy NGINX.....	15
3.3 Bascule en HTTPS et contrôles.....	18
3.4 Tunnel OpenVPN.....	19
3.4.1 Installation du serveur.....	19
3.4.2 Créer un accès client.....	20
3.4.3 Connexion et révocation.....	20
3.5 Pare-feu UFW.....	21
3.6 Isolation des conteneurs : la chaîne DOCKER-USER.....	22
3.6.1 Rendre la règle persistante — correctif prioritaire.....	22
3.7 Durcissement de GLPI.....	23
4. Exploitation courante.....	24

4.1 Commandes de base.....	24
4.2 Journalisation et diagnostic.....	24
4.3 Sauvegarde.....	24
4.4 Restauration.....	25
4.5 Renouvellement du certificat TLS.....	26
4.6 Mises à jour.....	27
5. État de conformité au cahier de recettes.....	28
5.1 Tableau de conformité.....	28
5.2 Poste client et agent d'inventaire.....	28
5.3 Collecte des courriels et création automatisée de tickets.....	29
5.4 Outil de contrôle à distance.....	29
5.5 Double authentification.....	29
5.6 Système de détection et de prévention d'intrusion Suricata.....	30
6. Registre des points de vigilance.....	31
Annexe A. docker-compose.yml.....	32
Annexe B. nginx.conf.....	32
Annexe C. Jeu de règles réseau de référence.....	33
Annexe D. Table des figures.....	33

Ce document est la référence unique du serveur de ticketing. Il s'adresse à toute personne appelée à l'installer, le sécuriser, l'exploiter ou le reconstruire. Les procédures des chapitres 2 et 3 sont complètes et reproductibles depuis un système nu, et illustrées par les captures relevées lors du déploiement initial : pour chaque étape, la commande, le résultat effectivement obtenu et le contrôle à effectuer. Toutes les commandes sont données telles qu'exécutées sur Ubuntu Server 22.04.

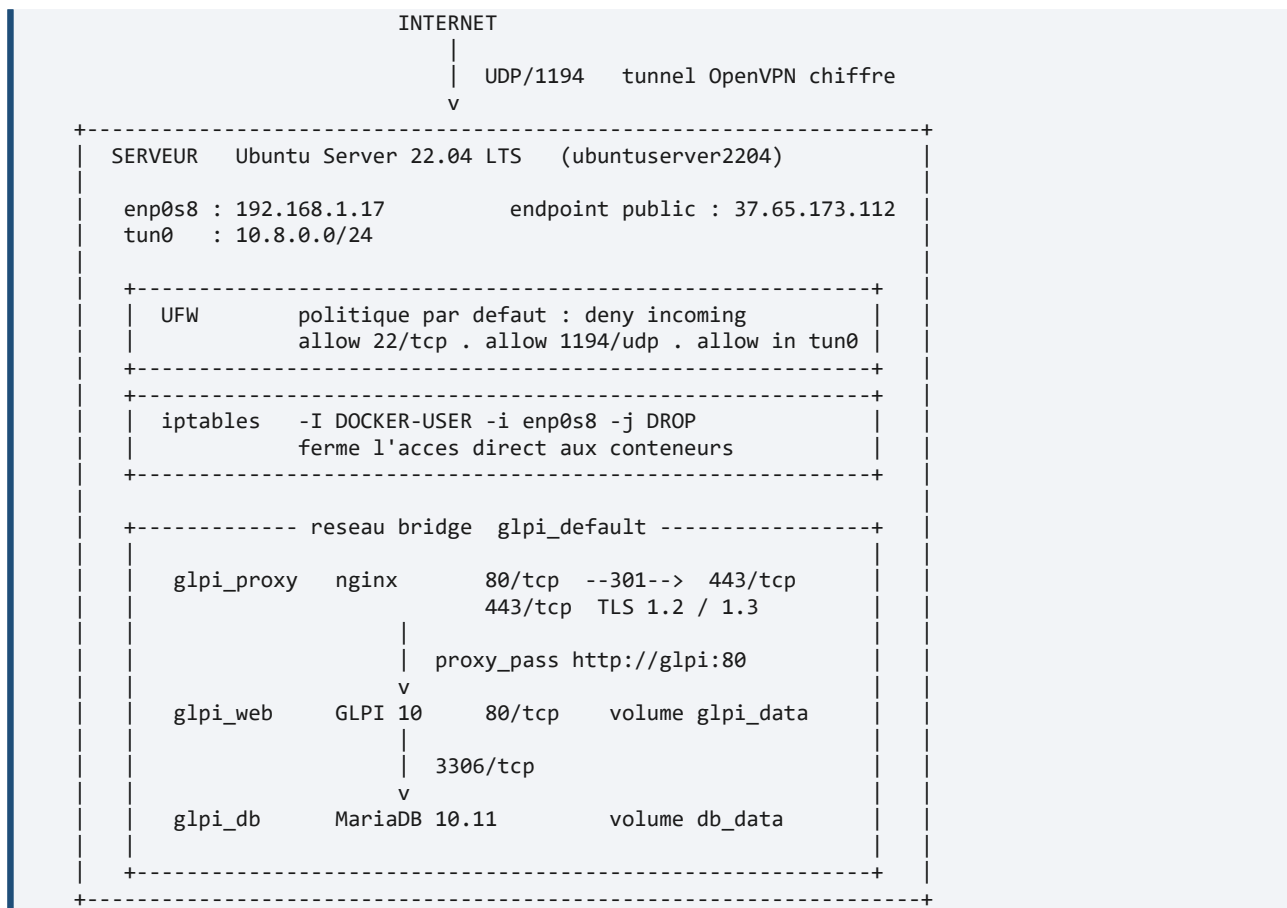
1. Présentation du service

1.1 Objet et périmètre

Le service met à disposition un outil de gestion de tickets et de parc informatique, GLPI, à destination des utilisateurs et des techniciens. Le cahier de recettes du projet ne demande pas seulement de l'installer, mais de le **rendre inaccessible autrement que par un tunnel VPN**. Deux de ses remarques commandent l'ensemble de l'architecture décrite ici : « *Attention à sécuriser le tunnel* » et « *N'autoriser que le tunnel du VPN en entrée pour passer* ».

Cette contrainte n'est pas un détail de configuration : elle explique pourquoi le port 443 n'est pas ouvert dans le pare-feu, pourquoi une règle **iptables** particulière est nécessaire, et pourquoi certaines options courantes – TeamViewer, par exemple – sont écartées au chapitre 5. Toute intervention sur ce serveur doit la préserver.

1.2 Architecture



Architecture du service : un seul point d'entrée, le tunnel VPN.

1.3 Inventaire de la plateforme

Élément	Valeur	Remarque
Système d'exploitation	Ubuntu Server 22.04 LTS (jammy)	Support jusqu'en avril 2027
Nom d'hôte	ubuntuserver2204	
Utilisateur d'administration	ubuntu	Élévation par sudo

Interface physique	enp0s8 – 192.168.1.17	Réseau local
Interface du tunnel	tun0 – 10.8.0.0/24	Créée par OpenVPN ; le serveur prend 10.8.0.1
Adresse publique	37.65.173.112	Point d'entrée du tunnel
Moteur de conteneurs	docker.io (dépôt Ubuntu)	
Orchestration	docker-compose 1.29.2 et docker compose v2	Les deux sont présents ; utiliser `docker compose` (v2)
Mémoire requise	4 Go minimum	Exigence du cahier de recettes
Répertoire de travail	/home/ubuntu/ticketing-projet	

1.4 Composants applicatifs et persistance

Conteneur	Image	Rôle	Ports publiés	Volume monté
glpi_proxy	nginx:latest	Terminaison TLS, redirection 80 vers 443, reverse proxy	80, 443	nginx.conf et certs/ en lecture seule
glpi_web	diouxx/glpi:latest	Application GLPI 10	aucun	glpi_data sur /var/www/html/glpi
glpi_db	mariadb:10.11	Base de données glpidb	aucun	db_data sur /var/lib/mysql

Le fait que **glpi_web** et **glpi_db** ne publient **aucun** port est structurant : même si le filtrage réseau venait à tomber, l'application et la base resteraient inatteignables autrement que par le proxy.

Réseau Docker : **glpi_default**, de type bridge, créé automatiquement par Compose. Les conteneurs s'y résolvent par leur **nom de service** – **db**, **glpi**, **nginx** – et non par leur nom de conteneur. C'est pourquoi **nginx.conf** relaie vers **http://glpi:80**.

Volume Docker	Contenu	Conséquence en cas de perte
glpi_db_data	Données MariaDB : parc, tickets, utilisateurs, droits	Perte totale de l'historique
glpi_glpi_data	Fichiers GLPI : documents joints, greffons, configuration locale	Perte des pièces jointes des tickets

Nom réel des volumes

Compose préfixe les volumes par le nom du répertoire du projet. Le fichier de composition étant dans **~/ticketing-projet/glpi/**, les volumes s'appellent **glpi_db_data** et **glpi_glpi_data** – et non **db_data** et **glpi_data**. C'est sous ces noms qu'il faut les désigner dans **docker volume ls** et dans les procédures de sauvegarde du paragraphe 4.3.

1.5 Matrice des flux réseau

Cette matrice décrit l'état attendu du filtrage. C'est la référence à consulter lors de tout dépannage d'accès.

#	Source	Destination	Port	Décision et règle applicable
---	--------	-------------	------	------------------------------

1	Internet	Serveur, <code>enp0s8</code>	1194/udp	Autorisé – <code>ufw allow 1194/udp</code>
2	Poste d'administration	Serveur, <code>enp0s8</code>	22/tcp	Autorisé – <code>ufw allow 22/tcp</code>
3	Client du tunnel, 10.8.0.0/24	<code>glpi_proxy</code>	443/tcp	Autorisé – <code>ufw allow in on tun0</code> ; la règle <code>DOCKER-USER</code> ne s'applique pas, l'interface d'entrée étant <code>tun0</code>
4	Client du tunnel	<code>glpi_proxy</code>	80/tcp	Autorisé puis redirigé en 301 vers 443/tcp par NGINX
5	Réseau local, 192.168.1.0/24	Conteneurs	tout	Rejeté – <code>DOCKER-USER -i enp0s8 -j DROP</code> . C'est la règle qui applique la consigne du projet.
6	Réseau local	Serveur, hors 22 et 1194	tout	Rejeté – <code>ufw default deny incoming</code>
7	<code>glpi_proxy</code>	<code>glpi_web</code>	80/tcp	Autorisé – réseau <code>glpi_default</code>
8	<code>glpi_web</code>	<code>glpi_db</code>	3306/tcp	Autorisé – réseau <code>glpi_default</code>
9	Serveur et conteneurs	Internet	tout	Autorisé – <code>ufw default allow outgoing</code> (mises à jour, futur collecteur de courriels)

Le port 443 n'est pas ouvert dans UFW, et c'est voulu

L'absence de `ufw allow 443/tcp` n'est pas un oubli. Le service ne doit être joignable que depuis le tunnel, où c'est la règle `allow in on tun0` qui l'autorise. **Ne pas ouvrir 443 pour dépanner un accès** : cela annulerait l'ensemble du dispositif. Si un client ne parvient pas à joindre GLPI, vérifier d'abord que son tunnel est monté – voir l'arbre de diagnostic du paragraphe 4.2.

1.6 Arborescence des fichiers

```
/home/ubuntu/ticketing-projet/
|
+-- glpi/
|   +-- .env                secrets MariaDB    (600, jamais versionne)
|   +-- docker-compose.yml  definition des trois services
|
+-- nginx/
    +-- nginx.conf          reverse proxy et terminaison TLS
    +-- certs/
        +-- glpi.crt        certificat auto-signé    (644)
        +-- glpi.key        cle privée                (600, root)
```

Chemins relatifs du fichier de composition

`docker-compose.yml` monte `../nginx/nginx.conf` et `../nginx/certs`. Ces chemins sont relatifs au répertoire du fichier, soit `~/ticketing-projet/glpi/` : ils sont donc **corrects sur le serveur**. Attention en revanche dans le dépôt Git, où les deux fichiers sont regroupés dans un unique répertoire `serveur/` : y déplier l'arborescence ci-dessus avant de lancer la pile.

1.7 Le principe retenu : la défense en profondeur

La sécurisation ne repose pas sur une mesure unique mais sur quatre couches indépendantes. Chacune répond à une menace précise, et la défaillance de l'une ne suffit pas à compromettre le service. Cette grille sert de fil conducteur au chapitre 3 et de référence lors de tout audit.

Couche	Mécanisme	Menace couverte	Procédure
1. Périmètre réseau	OpenVPN en 1194/udp, authentification par certificat (PKI)	Exposition directe du service de ticketing sur Internet	3.4
2. Filtrage réseau	UFW en <code>deny incoming</code> + règle <code>DOCKER-USER</code> bloquant <code>enp0s8</code>	Accès au service depuis le réseau local sans passer par le tunnel	3.5, 3.6
3. Transport	Certificat X.509, TLS 1.2 et 1.3, redirection 301 de HTTP vers HTTPS	Interception des identifiants et des données circulant en clair	3.1 à 3.3
4. Application	Mots de passe, double authentification, politique de mot de passe	Comptes par défaut de GLPI, publiquement documentés	2.8, 3.7

2. Procédure d'installation

Cette procédure installe le service depuis un Ubuntu Server 22.04 fraîchement déployé. Elle correspond à ce qui a été exécuté lors du déploiement initial, dont les captures constituent le résultat attendu de chaque étape. À l'issue du chapitre, GLPI est fonctionnel mais **encore accessible en HTTP et sans restriction réseau** : le chapitre 3 est indissociable de celui-ci.

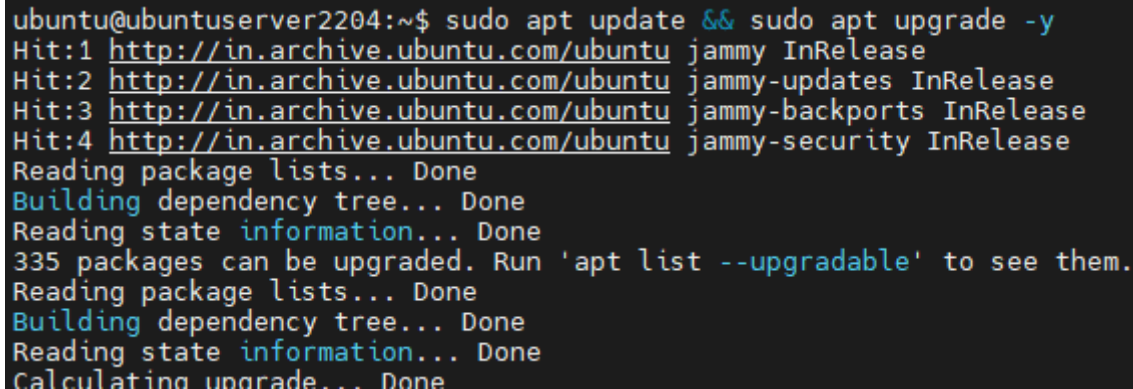
2.1 Prérequis

- Une machine virtuelle Ubuntu Server 22.04 LTS, **4 Go de RAM minimum**, avec accès administrateur **sudo**.
- Une interface réseau atteignable depuis le réseau local (ici **enp0s8**, 192.168.1.17) et une adresse publique pour le point d'entrée du tunnel.
- Un accès Internet sortant pour récupérer les paquets et les images de conteneurs.
- Un accès console à la machine, ou une seconde session SSH ouverte : plusieurs étapes du chapitre 3 manipulent le pare-feu et peuvent couper la session en cours.

2.2 Mise à jour du socle

Premier réflexe avant toute installation : appliquer les correctifs de sécurité en attente. Installer un service à sécuriser sur un système obsolète reviendrait à verrouiller la porte d'entrée en laissant les fenêtres ouvertes.

```
sudo apt update && sudo apt upgrade -y
sudo reboot # uniquement si un nouveau noyau a été installé
```



```
ubuntu@ubuntuserver2204:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
335 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
```

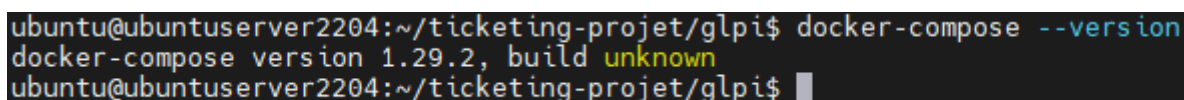
Figure 1 – Résultat attendu : la liste des paquets est actualisée et la mise à niveau démarre. Lors du déploiement initial, 335 paquets étaient en attente.

2.3 Installation de Docker

```
sudo apt install docker.io docker-compose -y
sudo systemctl enable --now docker
docker-compose --version
```

```
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ sudo apt install docker.io docker-compose -y
```

Figure 2 – Installation du moteur Docker et de Docker Compose.



```
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ docker-compose --version
docker-compose version 1.29.2, build unknown
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$
```

Figure 3 – Contrôle de la version. La commande doit répondre sans erreur ; la version 1.29.2 est celle des dépôts Ubuntu 22.04.

Optionnel, pour éviter `sudo` devant chaque commande Docker. La session doit être rouverte pour que l'appartenance au groupe prenne effet :

```
sudo usermod -aG docker $USER
```

Implication de sécurité

Appartenir au groupe `docker` équivaut à disposer des droits `root` sur la machine : un conteneur peut monter le système de fichiers de l'hôte. **N'y ajouter aucun compte de service ni utilisateur non administrateur.**

2.4 Arborescence et secrets

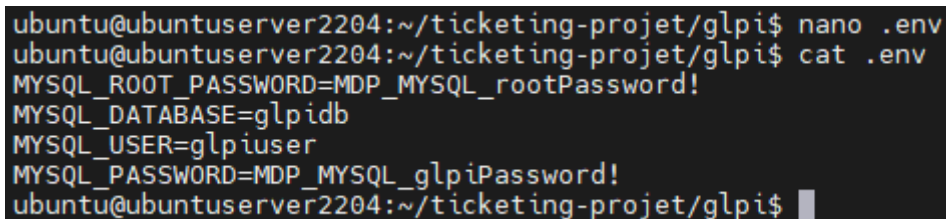
```
mkdir -p ~/ticketing-projet/glpi ~/ticketing-projet/nginx/certs
cd ~/ticketing-projet/glpi
nano .env
```

Les identifiants de la base sont placés dans un fichier `.env` que Docker Compose charge automatiquement. L'intérêt : `docker-compose.yml` peut être versionné et partagé, alors que `.env` reste local. Les mots de passe doivent être **générés, non choisis** :

```
MYSQL_ROOT_PASSWORD=<sortie de : openssl rand -base64 24>
MYSQL_DATABASE=glpidb
MYSQL_USER=glpiuser
MYSQL_PASSWORD=<sortie de : openssl rand -base64 24>

chmod 600 .env
echo '.env' >> ~/ticketing-projet/.gitignore
```

Le fichier `.env` ne doit être lisible que par son propriétaire, et jamais versionné.



```
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ nano .env
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ cat .env
MYSQL_ROOT_PASSWORD=MDP_MYSQL_rootPassword!
MYSQL_DATABASE=glpidb
MYSQL_USER=glpiuser
MYSQL_PASSWORD=MDP_MYSQL_glpiPassword!
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$
```

Figure 4 – Structure du fichier `.env` telle que créée lors du déploiement initial.

Point de vigilance

La capture ci-dessus affiche les mots de passe en clair. C'est acceptable sur un environnement de laboratoire éphémère, mais un `.env` ne doit jamais être versionné ni capturé sur un système réel. Les valeurs employées lors du déploiement initial doivent être considérées comme divulguées et **régénérées** – voir l'entrée 10 du registre du chapitre 6. Publier uniquement un `.env.example` sans valeurs.

2.5 Fichier de composition

Créer `~/ticketing-projet/glpi/docker-compose.yml`. À ce stade de la procédure, il ne déclare que deux services – la base et l'application – et publie le port 80 pour permettre l'accès à l'assistant d'installation.

```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ nano docker-compose.yml
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ cat docker-compose.yml
version: '3.8'

services:
  db:
    image: mariadb:10.11
    container_name: glpi_db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  glpi:
    image: diouxx/glpi:latest
    container_name: glpi_web
    restart: always
    ports:
      - "80:80"
    environment:
      - TIMEZONE=Europe/Paris
    depends_on:
      - db
    volumes:
      - glpi_data:/var/www/html/glpi

volumes:
  db_data:
  glpi_data:
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ █

```

Figure 5 – Fichier de composition initial : MariaDB et GLPI. Noter la section ports du service glpi, qui sera retirée au paragraphe 3.2.

Reconstruction depuis zéro : deux étapes ou une seule ?

Le déploiement initial s'est fait en deux temps – installer en HTTP, puis placer le proxy devant – et cette documentation suit le même ordre pour que chaque capture corresponde à son étape. **Pour une reconstruction, il est préférable d'appliquer directement le fichier complet de l'annexe A**, puis de dérouler le paragraphe 3.1 (certificat) avant de lancer la pile : l'assistant d'installation de GLPI fonctionne aussi bien à travers le proxy HTTPS, et le service n'est alors jamais exposé en clair, même temporairement.

2.6 Premier démarrage

```

cd ~/ticketing-projet/glpi
sudo docker compose up -d
sudo docker compose ps

```

```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ sudo docker-compose up -d
Creating network "glpi_default" with the default driver
Creating volume "glpi_db_data" with default driver
Creating volume "glpi_glpi_data" with default driver
Pulling db (mariadb:10.11)...
10.11: Pulling from library/mariadb
0bf9ac32a939: Pull complete
92ff57b1eb9b: Pull complete
d544298cabd5: Pull complete
c0b110f56249: Pull complete
3c10d7f0cd20: Pull complete
38a738597cc9: Pull complete
8bee7ee7d49c: Pull complete
758b07585e0a: Pull complete
f8b370294f64: Download complete
db02198483d1: Download complete
Digest: sha256:ce66c7be32a03aabe7241d0a10993a2db827ef652a35d25727d92a832ac8ef73
Status: Downloaded newer image for mariadb:10.11
Pulling glpi (diouxx/glpi:latest)...
latest: Pulling from diouxx/glpi
0bb5c2a7dd90: Pull complete
9821c61dc279: Pull complete
fea1432adf09: Pull complete
b9347fe14756: Pull complete
ece6a8fdeb39: Download complete
Digest: sha256:9ca43af63ddcb7e1c9b7b452a50b8204980ac549526dee1c00296793bfe7789f
Status: Downloaded newer image for diouxx/glpi:latest
Creating glpi_db ... done
Creating glpi_web ... done

```

Figure 6 – Résultat attendu : création du réseau et des volumes, téléchargement des images, puis démarrage des conteneurs.

```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ sudo docker-compose ps
Name                                Command                                State      Ports
-----
glpi_db    docker-entrypoint.sh mariadb         Up         3306/tcp
glpi_web   /opt/glpi-start.sh                  Up         443/tcp, 0.0.0.0:80->80/tcp, :::80->80/tcp
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$

```

Figure 7 – Les deux conteneurs sont à l'état Up. glpi_db n'expose 3306 qu'en interne, tandis que glpi_web publie 0.0.0.0:80 – situation corrigée au paragraphe 3.2.

2.7 Assistant d'installation de GLPI

Ouvrir <http://<adresse du serveur>/> et dérouler l'assistant en six étapes. Paramètres de connexion à la base, à saisir tels quels :

Champ de l'assistant	Valeur	Origine
Serveur SQL	db	Nom du service dans le fichier de composition, pas son nom de conteneur
Utilisateur SQL	glpiuser	MYSQL_USER du fichier .env
Mot de passe SQL	valeur de MYSQL_PASSWORD	Fichier .env
Base de données	glpidb	MYSQL_DATABASE du fichier .env



Figure 8 – Étape 6 : l'installation est terminée. GLPI énumère les quatre comptes créés par défaut.

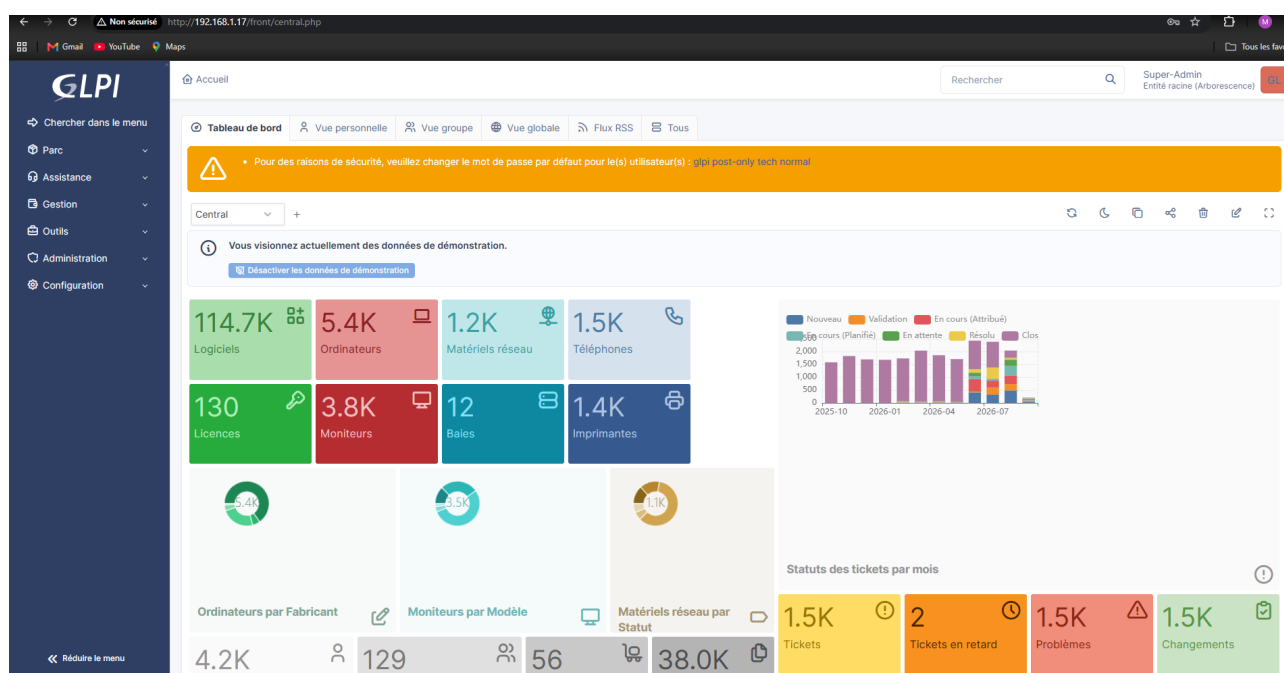


Figure 9 – Première connexion. Trois points à traiter sans délai : l'accès est en HTTP, le navigateur signale « Non sécurisé », et GLPI réclame le changement des mots de passe de glpi, post-only, tech et normal.

2.8 Durcissement immédiat des comptes

Les quatre comptes créés par l'assistant constituent la première vulnérabilité concrète du déploiement : leurs identifiants figurent dans la documentation publique de GLPI et sont testés systématiquement par les outils de balayage automatisé.

Compte	Mot de passe par défaut	Profil	État au déploiement initial
glpi	glpi	Super-Admin	Traité – voir figure 10
tech	tech	Technicien	À traiter
normal	normal	Normal	À traiter

post-only	postonly	Self-Service	À traiter

Depuis *Administration > Utilisateurs*, pour chacun des quatre comptes :

Figure 10 – Changement du mot de passe du compte Super-Admin glpi.

Contrôle de l'efficacité du changement

GLPI affiche en tête du tableau de bord la liste des comptes dont le mot de passe est resté par défaut. En figure 9, avant l'opération, cette liste comporte quatre noms : *glpi*, *post-only*, *tech*, *normal*. En figure 15, après le changement, elle n'en comporte plus que trois : *post-only*, *tech*, *normal*. **La disparition d'un compte de ce bandeau est le contrôle à utiliser** pour vérifier qu'un mot de passe a bien été pris en compte, et pour suivre ce qui reste à traiter.

Deux autres actions relèvent du même moment :

```
# 1. desactiver les donnees de demonstration
#   depuis le tableau de bord : bouton "Desactiver les donnees de demonstration"
#   (sinon 5 400 ordinateurs et 1 500 tickets fictifs faussent tout indicateur)

# 2. supprimer le fichier d'installation, qui reste accessible par le web
sudo docker exec glpi_web rm -f /var/www/html/glpi/install/install.php
```

Pourquoi supprimer `install.php`

Tant que ce fichier est présent, l'assistant d'installation reste joignable par le web et permet, dans certaines configurations, de réinitialiser la connexion à la base. Sa suppression est une étape standard de durcissement de

GLPI, à refaire après chaque mise à jour de l'image puisqu'elle porte sur le contenu du conteneur.

2.9 Contrôles de fin d'installation

```
cd ~/ticketing-projet/glpi

# les deux conteneurs sont a l'etat Up
sudo docker compose ps

# l'application repond
curl -I http://localhost/

# la base accepte les connexions
sudo docker exec glpi_db mariadb-admin ping
```



```

ubuntu@ubuntu2204:~/ticketing-projet/nginx$ nano nginx.conf
ubuntu@ubuntu2204:~/ticketing-projet/nginx$ cat nginx.conf
server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/nginx/certs/glpi.crt;
    ssl_certificate_key /etc/nginx/certs/glpi.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://glpi:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
ubuntu@ubuntu2204:~/ticketing-projet/nginx$ █

```

Figure 12 – Configuration du reverse proxy.

Ajouter ensuite le service **nginx** au fichier de composition, sous le nom de conteneur **glpi_proxy**. **Le point déterminant est ce qui disparaît** : la section **ports** du service **glpi** est supprimée. Sans cette suppression, le port 80 du conteneur GLPI resterait publié et offrirait un accès en clair contournant entièrement le proxy et son chiffrement.


```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ nano docker-compose.yml
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ cat docker-compose.yml
version: '3.8'

services:
  db:
    image: mariadb:10.11
    container_name: glpi_db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  glpi:
    image: diouxx/glpi:latest
    container_name: glpi_web
    restart: always
    environment:
      - TIMEZONE=Europe/Paris
    depends_on:
      - db
    volumes:
      - glpi_data:/var/www/html/glpi

  nginx:
    image: nginx:latest
    container_name: glpi_proxy
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ../nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ../nginx/certs:/etc/nginx/certs:ro
    depends_on:
      - glpi

volumes:
  db_data:
  glpi_data:
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ █

```

Figure 13 – Fichier de composition final. Le service `glpi` ne comporte plus de section `ports` : comparer avec la figure 5.

Élément du fichier	Pourquoi il est ainsi
Le service <code>glpi</code> n'a aucune section <code>ports</code>	Un port publié sur <code>glpi</code> offrirait un accès HTTP en clair contournant NGINX. C'est la principale erreur à ne pas réintroduire.
Le service <code>db</code> n'a aucune section <code>ports</code>	MariaDB ne doit être joignable que depuis le réseau des conteneurs, jamais depuis l'hôte ni le réseau local.
<code>nginx</code> monte ses fichiers en <code>:ro</code>	Lecture seule : un conteneur compromis ne peut pas réécrire la configuration du proxy ni remplacer le certificat.

3.3 Bascule en HTTPS et contrôles

```
cd ~/ticketing-projet/glpi
sudo docker compose down
sudo docker compose up -d
```

```
ubuntu@ubuntu2204:~/ticketing-projet/glpi$ sudo docker compose down
WARN[0000] /home/ubuntu/ticketing-projet/glpi/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  ✓ Container glpi_proxy   Removed
  ✓ Container glpi_web     Removed
  ✓ Container glpi_db      Removed
  ✓ Network glpi_default   Removed
ubuntu@ubuntu2204:~/ticketing-projet/glpi$ sudo docker compose up -d
WARN[0000] /home/ubuntu/ticketing-projet/glpi/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  ✓ Network glpi_default   Created
  ✓ Container glpi_db      Started
  ✓ Container glpi_web     Started
  ✓ Container glpi_proxy   Started
ubuntu@ubuntu2204:~/ticketing-projet/glpi$
```

Figure 14 – Résultat attendu : quatre éléments démarrés – le réseau et les trois conteneurs.

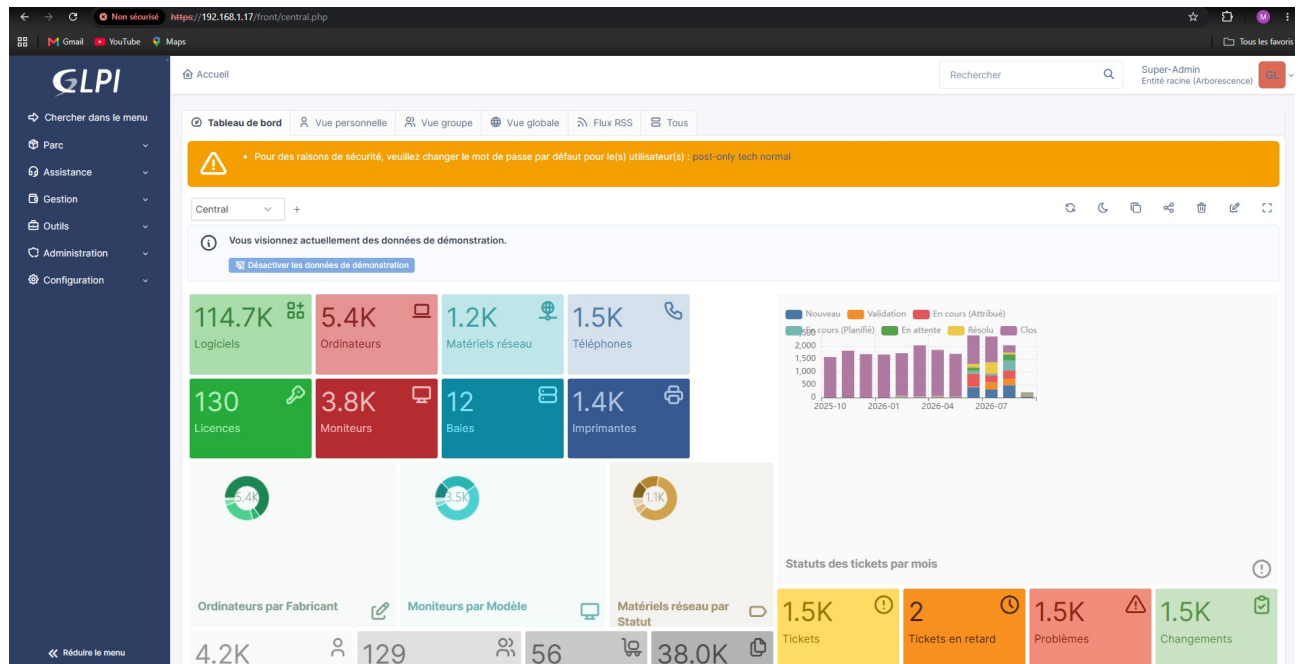


Figure 15 – GLPI répond en HTTPS. L'avertissement du navigateur est attendu : le certificat est auto-signé.

L'avertissement « Non sécurisé » subsiste et doit être interprété correctement : le **chiffrement du transport est bien effectif**, seule l'**authentification du serveur** n'est pas vérifiable, faute de signature par une autorité reconnue. Sa levée passe par une autorité de certification interne – voir l'entrée 5 du registre du chapitre 6.

Contrôles à effectuer dans l'ordre :

```
# 1. les trois conteneurs sont Up, et seul glpi_proxy publie 80 et 443
sudo docker compose ps

# 2. la configuration NGINX est syntaxiquement valide
sudo docker exec glpi_proxy nginx -t

# 3. le port 80 renvoie bien une redirection 301
curl -I http://localhost/

# 4. le port 443 répond (-k : on accepte le certificat auto-signé)
curl -Ik https://localhost/

# 5. le certificat présente les bons noms et la bonne échéance
echo | openssl s_client -connect localhost:443 2>/dev/null \
| openssl x509 -noout -subject -ext subjectAltName -enddate
```

3.4 Tunnel OpenVPN

C'est la couche qui applique la consigne centrale du cahier de recettes. Plutôt qu'exposer GLPI sur Internet, le service devient joignable uniquement depuis l'intérieur d'un tunnel chiffré.

3.4.1 Installation du serveur

L'installation s'appuie sur le script de déploiement OpenVPN d'angristan, largement utilisé, qui automatise la mise en place de la PKI.

```
cd ~
curl -O https://raw.githubusercontent.com/angristan/openvpn-install/master/openvpn-install.sh
chmod +x openvpn-install.sh
sudo ./openvpn-install.sh install
```

```
ubuntu@ubuntu204:~$ curl -O https://raw.githubusercontent.com/angristan/openvpn-install/master/openvpn-install.sh
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 170k 100 170k    0     0  321k    0 --:--:-- --:--:-- --:--:-- 321k
ubuntu@ubuntu204:~$ chmod +x openvpn-install.sh
ubuntu@ubuntu204:~$ sudo ./openvpn-install.sh
OpenVPN installer and manager

Usage: openvpn-install <command> [options]

Commands:
install      Install and configure OpenVPN server
uninstall    Remove OpenVPN server
client       Manage client certificates
server       Server management
interactive  Launch interactive menu

Global Options:
--verbose    Show detailed output
--log <path> Log file path (default: openvpn-install.log)
--no-log     Disable file logging
--no-color   Disable colored output
-h, --help   Show help

Run 'openvpn-install <command> --help' for command-specific help.
ubuntu@ubuntu204:~$ sudo ./openvpn-install.sh install
[INFO] === OpenVPN Non-Interactive Install ===
[INFO] Running in non-interactive mode with the following settings:
[INFO] ENDPOINT=37.65.173.112
[INFO] ENDPOINT_TYPE=4
[INFO] CLIENT_IPV4=y
[INFO] CLIENT_IPV6=n
[INFO] VPN_SUBNET_IPV4=10.8.0.0
[INFO] VPN_SUBNET_IPV6=fd42:42:42::
[INFO] ROUTE_INTERNET=y
[INFO] CLIENT_TO_CLIENT=n
[INFO] LOCAL_NETWORKS=none
[INFO] PORT=1194
[INFO] PROTOCOL=udp
[INFO] DNS=cloudflare
[INFO] MULTI_CLIENT=n
[INFO] AUTH_MODE=pki
[INFO] CLIENT=client
[INFO] CLIENT_CERT_DURATION_DAYS=3650
[INFO] SERVER_CERT_DURATION_DAYS=3650
[INFO] Setting up official OpenVPN repository...
> apt-get update
> apt-get install -y ca-certificates curl
> mkdir -p /etc/apt/keyrings
> curl -fsSL https://swupdate.openvpn.net/repos/openvpn-repo-public.gpg -o /etc/apt/keyrings/openvpn-repo-public.asc
[INFO] Updating package lists with new repository...
> apt-get update
[INFO] OpenVPN official repository configured
[INFO] Installing OpenVPN and dependencies...
> apt-get install -y openvpn iptables openssl curl ca-certificates tar dnsutils socat
```

Figure 16 – Installation en mode non interactif. Le script affiche les paramètres retenus avant de configurer le dépôt officiel OpenVPN et d'installer les dépendances.

Paramètres appliqués lors du déploiement initial :

Paramètre	Valeur	Signification
ENDPOINT / ENDPOINT_TYPE	37.65.173.112 / 4	Adresse IPv4 publique annoncée aux clients
PORT / PROTOCOL	1194 / udp	Écoute du serveur ; UDP réduit la latence du tunnel
VPN_SUBNET_IPV4	10.8.0.0/24	Plage attribuée aux clients ; le serveur prend 10.8.0.1
AUTH_MODE	pki	Authentification par certificat client, et non par mot de passe partagé

CLIENT_TO_CLIENT	n	Bon réglage : les clients ne peuvent pas se joindre entre eux, ce qui limite les déplacements latéraux si un poste est compromis
ROUTE_INTERNET	y	L'intégralité du trafic client passe par le tunnel
DNS	cloudflare	Résolution via le tunnel, sans fuite de requêtes
CLIENT_IPV6	n	IPv6 désactivé côté client, ce qui évite un contournement du filtrage IPv4
SERVER_CERT_DURATION_DAYS	3650	À réduire : dix ans de validité
CLIENT_CERT_DURATION_DAYS	3650	À réduire : dix ans de validité

3.4.2 Créer un accès client

Le script gère les certificats clients par sa sous-commande **client**. La syntaxe exacte des options dépend de la version téléchargée : la vérifier avant usage, ou passer par le menu interactif qui reste le chemin le plus sûr.

```
cd ~
sudo ./openvpn-install.sh client --help    # syntaxe exacte de la version installée
sudo ./openvpn-install.sh interactive      # menu : ajouter / révoquer un client
```

Le profil **.ovpn** généré contient la clé et le certificat du client : **c'est un secret complet d'accès au réseau**. Il est écrit dans le répertoire personnel de **root**. Transmission puis nettoyage :

```
# sur le serveur : rendre le profil récupérable par l'utilisateur ubuntu
sudo cp /root/poste-technicien.ovpn /home/ubuntu/
sudo chown ubuntu:ubuntu /home/ubuntu/poste-technicien.ovpn

# depuis le poste client : récupération par un canal chiffré
scp ubuntu@192.168.1.17:/home/ubuntu/poste-technicien.ovpn .

# sur le serveur : effacer la copie intermédiaire
shred -u /home/ubuntu/poste-technicien.ovpn
```

Un profil par personne, jamais de profil partagé

Émettre un profil distinct par utilisateur est ce qui rend la révocation possible : retirer l'accès à une personne sans reconfigurer tout le monde. Un profil partagé entre plusieurs techniciens interdit toute révocation ciblée et rend les journaux de connexion inexploitable.

3.4.3 Connexion et révocation

```
# poste client Linux
sudo apt install openvpn -y
sudo openvpn --config poste-technicien.ovpn

# vérification, une fois le tunnel monte
ip addr show tun0
ping -c 3 10.8.0.1

# révocation d'un accès, sur le serveur
sudo ./openvpn-install.sh interactive    # puis : révoquer un client
```

3.5 Pare-feu UFW

Le tunnel n'a de valeur que si le service n'est pas également joignable par un autre chemin. Le pare-feu est configuré en refus par défaut, puis ouvert au strict nécessaire.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 1194/udp
sudo ufw allow in on tun0
sudo ufw enable
sudo ufw status verbose
```

```
ubuntu@ubuntuserver2204:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
ubuntu@ubuntuserver2204:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
ubuntu@ubuntuserver2204:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
ubuntu@ubuntuserver2204:~$ sudo ufw allow 1194/udp
Rules updated
Rules updated (v6)
ubuntu@ubuntuserver2204:~$ sudo ufw allow in on tun0
Rules updated
Rules updated (v6)
ubuntu@ubuntuserver2204:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
ubuntu@ubuntuserver2204:~$
```

Figure 17 – Mise en place du jeu de règles. La demande de confirmation de `ufw enable` avertit que l'opération peut interrompre les connexions SSH existantes.

Ordre des commandes

Ouvrir 22/tcp **avant** d'activer le pare-feu. L'inverse coupe la session SSH en cours et rend le serveur inaccessible s'il n'y a pas d'accès console. C'est précisément ce que signale l'avertissement affiché par `ufw enable`.

Règle	Justification
<code>default deny incoming</code>	Tout ce qui n'est pas explicitement autorisé est refusé : c'est la posture correcte, l'inverse d'une liste noire toujours incomplète
<code>default allow outgoing</code>	Le serveur doit pouvoir sortir pour ses mises à jour et, à terme, pour interroger la boîte de courriels du collecteur
<code>allow 22/tcp</code>	Administration SSH ; seule ouverture directe conservée
<code>allow 1194/udp</code>	Point d'entrée du tunnel OpenVPN
<code>allow in on tun0</code>	Tout ce qui arrive par le tunnel est accepté. C'est cette règle qui rend GLPI accessible aux clients VPN
443/tcp non ouvert	Omission délibérée, cœur de la consigne : HTTPS n'est pas joignable depuis l'extérieur du tunnel

État attendu de `ufw status verbose` :

```
Status: active
Default: deny (incoming), allow (outgoing), disabled (routed)

To          Action      From
--          -
22/tcp      ALLOW IN    Anywhere
1194/udp    ALLOW IN    Anywhere
Anywhere on tun0 ALLOW IN    Anywhere
```

3.6 Isolation des conteneurs : la chaîne DOCKER-USER

Le problème. Le jeu de règles précédent ne suffit pas. Docker publie les ports de ses conteneurs en écrivant directement dans `iptables` : une règle de traduction d'adresse dans la table `nat`, et une autorisation de transit dans la chaîne `FORWARD`. Ces règles sont évaluées **avant** celles que gère UFW, lequel travaille sur `INPUT`. Conséquence : **un port publié par un conteneur reste joignable malgré un pare-feu en `deny incoming`**. Le pare-feu n'est pas contourné par malveillance, il n'est simplement jamais consulté pour ce trafic, qui est routé et non destiné à l'hôte. C'est une cause récurrente d'exposition involontaire de services conteneurisés.

La solution. Docker réserve la chaîne `DOCKER-USER` à l'administrateur : il la traverse avant d'appliquer ses propres règles et ne la réécrit jamais au redémarrage du démon.

```
sudo iptables -I DOCKER-USER -i enp0s8 -j DROP
```

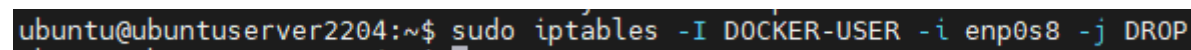


Figure 18 – Blocage de tout trafic à destination des conteneurs arrivant par l'interface physique.

Le filtre porte sur l'**interface d'entrée**, ce qui sépare exactement les deux chemins d'accès :

Trafic	Interface d'entrée	Résultat
Poste du réseau local vers l'interface web de GLPI	<code>enp0s8</code>	Rejeté par la règle <code>DOCKER-USER</code>
Client connecté au tunnel VPN vers l'interface web de GLPI	<code>tun0</code>	Accepté : la règle ne s'applique pas, le paquet atteint NGINX

C'est cette règle, et non le pare-feu seul, qui rend la consigne « n'autoriser que le tunnel du VPN en entrée » réellement effective.

```
# verification : la regle doit apparaitre en position 1
sudo iptables -L DOCKER-USER -n -v --line-numbers
```

3.6.1 Rendre la règle persistante — correctif prioritaire

Une unité `systemd` est préférable à `iptables-persistent` dans ce cas précis : elle garantit que la règle est posée **après** le démarrage du démon Docker, donc après que celui-ci a recréé la chaîne. Avec `iptables-persistent` seul, la restauration peut intervenir avant Docker et être effacée par lui.

```

sudo tee /etc/systemd/system/docker-isolation.service > /dev/null <<'EOF'
[Unit]
Description=Isolation des conteneurs Docker vis-a-vis du reseau local
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/sbin/iptables -I DOCKER-USER -i enp0s8 -j DROP
ExecStop=/usr/sbin/iptables -D DOCKER-USER -i enp0s8 -j DROP

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now docker-isolation.service
sudo systemctl status docker-isolation.service

```

Recette du correctif, à exécuter pour valider qu'il fonctionne réellement :

```

# 1. redemarrer le serveur
sudo reboot

# 2. au retour, la regle doit etre presente sans intervention
sudo iptables -L DOCKER-USER -n --line-numbers

# 3. depuis un poste du reseau local, HORS tunnel : doit ECHOUER
curl -Ik --max-time 5 https://192.168.1.17/

# 4. depuis un poste avec le tunnel monte : doit REPONDRE
curl -Ik --max-time 5 https://10.8.0.1/

```

Les étapes 3 et 4 sont le seul contrôle qui atteste vraiment de l'isolation. **Lire le jeu de règles ne suffit pas** : c'est exactement l'erreur qui laisse un service exposé malgré un pare-feu correctement configuré. Toute intervention touchant au réseau, au pare-feu ou à Docker doit se terminer par ces deux commandes.

3.7 Durcissement de GLPI

Mesure	Emplacement dans GLPI	État actuel
Mot de passe du compte glpi	Administration > Utilisateurs	Fait
Mots de passe de tech , normal , post-only	Administration > Utilisateurs	À faire – identifiants par défaut publics
Désactivation des données de démonstration	Tableau de bord, bouton dédié	À faire – 5 400 ordinateurs et 1 500 tickets fictifs
Double authentification TOTP	Configuration > Authentification	À faire
Politique de mot de passe (longueur, complexité, expiration)	Configuration > Générale > Sécurité	À faire
Suppression de install/install.php	Système de fichiers du conteneur	À vérifier – et à refaire après chaque mise à jour d'image
Journalisation des connexions	Administration > Journaux	Actif par défaut

4. Exploitation courante

4.1 Commandes de base

Toutes les commandes `docker compose` se lancent depuis `~/ticketing-projet/glpi`, où se trouve le fichier de composition.

```
cd ~/ticketing-projet/glpi

sudo docker compose ps          # etat des trois conteneurs
sudo docker compose up -d       # demarrer, ou appliquer une modification
sudo docker compose restart nginx # redemarrer un seul service
sudo docker compose stop        # arreter sans supprimer
sudo docker compose down        # arreter et supprimer conteneurs et reseau
                                # (les volumes nommes sont conserves)
docker stats --no-stream        # consommation memoire et processeur
```

4.2 Journalisation et diagnostic

```
sudo docker compose logs -f --tail=100 # les trois services
sudo docker compose logs -f nginx      # un seul service
sudo docker exec glpi_proxy nginx -t   # valide de la configuration NGINX
sudo journalctl -u openvpn-server@server -f # tunnel VPN
sudo ufw status verbose                # regles du pare-feu
sudo iptables -L DOCKER-USER -n --line-numbers # isolation des conteneurs
```

Arbre de décision pour un client qui ne joint pas GLPI :

Symptôme	Vérification	Cause probable
Aucune réponse, délai d'attente dépassé	<code>ip addr show tun0</code> sur le poste client	Le tunnel n'est pas monté – c'est le cas le plus fréquent
Le tunnel est monté mais aucune réponse	<code>ping 10.8.0.1</code> depuis le client	Règle <code>allow in on tun0</code> absente, ou service OpenVPN arrêté
<code>ping</code> passe mais HTTPS échoue	<code>sudo docker compose ps</code> sur le serveur	Conteneur <code>glpi_proxy</code> arrêté
Erreur 502 renvoyée par NGINX	<code>sudo docker compose logs glpi</code>	<code>glpi_web</code> est arrêté ou en cours de démarrage
GLPI signale une erreur de base de données	<code>sudo docker compose logs db</code>	<code>glpi_db</code> est arrêté, ou volume <code>glpi_db_data</code> corrompu
Le service répond depuis le réseau local	<code>sudo iptables -L DOCKER-USER -n</code>	Anomalie de sécurité : la règle d'isolation a disparu, typiquement après un redémarrage. Appliquer le paragraphe 3.6.1

4.3 Sauvegarde

Aucune sauvegarde n'est en place à ce jour. Le script ci-dessous couvre les trois éléments dont la perte serait irréversible : la base, les fichiers de l'application, la configuration et les secrets.


```

sudo tee /usr/local/sbin/sauvegarde-glpi.sh > /dev/null <<'EOF'
#!/bin/bash
set -euo pipefail

DEST=/var/backups/glpi
STAMP=$(date +%F)
PROJET=/home/ubuntu/ticketing-projet
mkdir -p "$DEST"

# 1. base de donnees : --single-transaction assure un dump coherent
#   sans verrouiller les tables. Le mot de passe est lu dans
#   l'environnement du conteneur, il n'apparaît ni dans la ligne
#   de commande ni dans l'historique du shell.
docker exec glpi_db sh -c \
  'exec mariadb-dump --single-transaction -u root -p"$MYSQL_ROOT_PASSWORD" glpidb' \
  | gzip > "$DEST/glpidb-$STAMP.sql.gz"

# 2. fichiers de l'application : documents joints, greffons, configuration
docker run --rm -v glpi_glpi_data:/data -v "$DEST":/backup alpine \
  tar czf "/backup/glpi_data-$STAMP.tar.gz" -C /data .

# 3. configuration, secrets et certificats
tar czf "$DEST/config-$STAMP.tar.gz" -C "$PROJET" \
  glpi/.env glpi/docker-compose.yml nginx/

# 4. rotation : conserver quatorze jours
find "$DEST" -name '*.gz' -mtime +14 -delete
EOF

sudo chmod 700 /usr/local/sbin/sauvegarde-glpi.sh

```

Planification quotidienne à 2 h 00 :

```

echo '0 2 * * * /usr/local/sbin/sauvegarde-glpi.sh >> /var/log/sauvegarde-glpi.log 2>&1' \
  | sudo tee /etc/cron.d/sauvegarde-glpi > /dev/null
sudo chmod 644 /etc/cron.d/sauvegarde-glpi

```

Une sauvegarde non testée n'est pas une sauvegarde

Deux exigences complètent le script. **Copier les archives hors de la machine virtuelle** : conservées sur le serveur, elles disparaissent avec lui, ce qui est précisément le scénario contre lequel on se protège. Et **rejouer une restauration complète au moins une fois**, en suivant le paragraphe 4.4 : c'est le seul moyen de savoir que les archives sont exploitables. Noter enfin que **config-*.tar.gz** contient le fichier **.env** en clair et doit être protégé en conséquence.

4.4 Restauration

Procédure destructive

Les commandes ci-dessous **suppriment les volumes existants** avant de réinjecter les données. À n'exécuter que sur un serveur dont l'état actuel est perdu ou volontairement abandonné.

```

cd ~/ticketing-projet/glpi
STAMP=AAAA-MM-JJ          # date de l'archive a restaurer

# 1. arreter la pile et repartir de volumes vierges
sudo docker compose down
docker volume rm glpi_glpi_data glpi_db_data

# 2. demarrer la seule base, et attendre qu'elle accepte les connexions
sudo docker compose up -d db
until docker exec glpi_db mariadb-admin ping --silent 2>/dev/null; do sleep 2; done

# 3. reinjecter le dump
zcat "/var/backups/glpi/glpidb-$STAMP.sql.gz" \
| docker exec -i glpi_db sh -c 'exec mariadb -u root -p"$MYSQL_ROOT_PASSWORD" glpidb'

# 4. restaurer les fichiers de l'application
docker run --rm -v glpi_glpi_data:/data -v /var/backups/glpi:/backup alpine \
tar xzf "/backup/glpi_data-$STAMP.tar.gz" -C /data

# 5. remonter l'ensemble et verifier
sudo docker compose up -d
sudo docker compose ps
curl -Ik https://localhost/

```

Après restauration, contrôler dans l'interface que le nombre de tickets et d'éléments de parc correspond à l'attendu, et que les pièces jointes d'un ticket connu s'ouvrent correctement – ce second point valide la restauration du volume `glpi_glpi_data`, qu'un simple décompte ne couvre pas.

4.5 Renouvellement du certificat TLS

Le certificat est valable **365 jours** à compter de sa génération, et rien n'alerte de son expiration : passé l'échéance, les navigateurs refuseront la connexion. Relever l'échéance et la porter à l'agenda d'exploitation.

```

# relever la date d'expiration
openssl x509 -in ~/ticketing-projet/nginx/certs/glpi.crt -noout -enddate

# renouveler (memes parametres, subjectAltName inclus)
cd ~/ticketing-projet/nginx
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/glpi.key -out certs/glpi.crt \
-subj "/C=FR/ST=Loire/L=Nantes/O=MonEntreprise/OU=IT/CN=glpi.local" \
-addext "subjectAltName=DNS:glpi.local,IP:192.168.1.17,IP:10.8.0.1"
sudo chmod 600 certs/glpi.key

# recharger NGINX : un restart suffit, les fichiers sont montes depuis l'hote
cd ~/ticketing-projet/glpi
sudo docker compose restart nginx

```

Surveillance automatique de l'échéance, avec alerte à trente jours :

```

echo '0 8 * * 1 openssl x509 -in /home/ubuntu/ticketing-projet/nginx/certs/glpi.crt \
-noout -checkend 2592000 || echo "Certificat GLPI : expiration dans moins de 30 jours"' \
| sudo tee /etc/cron.d/verif-cert-glpi > /dev/null

```

4.6 Mises à jour

```
# systeme d'exploitation
sudo apt update && sudo apt upgrade -y

# images des conteneurs
cd ~/ticketing-projet/glpi
sudo /usr/local/sbin/sauvegarde-glpi.sh      # IMPERATIF avant toute mise a jour
sudo docker compose pull
sudo docker compose up -d
sudo docker image prune -f

# apres toute mise a jour de l'image GLPI, refaire :
sudo docker exec glpi_web rm -f /var/www/html/glpi/install/install.php
sudo iptables -L DOCKER-USER -n --line-numbers  # controler l'isolation
```

Les étiquettes `latest` sont un risque de mise à jour non maîtrisée

`diouxx/glpi:latest` et `nginx:latest` ne désignent pas une version : un `docker compose pull` peut donc livrer une **version majeure** de GLPI, avec migration de schéma de base irréversible et greffons rendus incompatibles.

Deux conséquences pratiques : ne jamais lancer `pull` sans sauvegarde vérifiée, et remplacer ces étiquettes par des versions explicites – à l'image de `mariadb:10.11`, qui est correctement épinglé.

5. État de conformité au cahier de recettes

Ce chapitre situe l'état du service par rapport aux exigences du cahier de recettes, et décrit la voie de mise en œuvre de ce qui reste à faire. Il complète le registre d'écarts du chapitre 6, qui porte sur les défauts de l'existant.

5.1 Tableau de conformité

Exigence du cahier de recettes	Priorité	État	Preuve	Renvoi
Environnement réseau : 2 machines, serveur et client	Très haute	Partiel	Fig. 1, 16	Serveur opérationnel ; le poste client existe comme client du tunnel, son paramétrage n'est pas attesté – voir 5.2
Serveur GLPI sur Ubuntu Server, 4 Go de RAM	Haute	Conforme	Fig. 5 à 9	Chapitre 2
Ajout d'un VPN pour se connecter à GLPI	Haute	Conforme	Fig. 16	3.4
Sécurisation du tunnel : n'autoriser que le VPN en entrée	Haute	Conforme	Fig. 17, 18	3.5 et 3.6 – non persistant , voir 3.6.1
Ajout d'un pare-feu pour protéger le réseau	Haute	Conforme	Fig. 17	3.5
Chiffrement des flux (<i>ajout hors cahier</i>)	–	Conforme	Fig. 11 à 15	3.1 à 3.3 – certificat auto-signé
Client et agent GLPI pour l'inventaire	Haute	À faire	–	5.2
Script de collecte des mails vers GLPI	Haute	À faire	–	5.3
Outil externe de contrôle à distance	Moyenne	À faire	–	5.4
Double authentification (MFA)	Haute	À faire	–	5.5
IPS Suricata	Haute	À faire	–	5.6

L'ensemble de la chaîne de sécurisation réseau demandée est en place et vérifiée : tunnel VPN, pare-feu, isolation des conteneurs, chiffrement du transport. Ce qui reste relève pour l'essentiel de paramétrage applicatif dans GLPI – trois des cinq points ci-dessous sont des fonctions natives de la version 10, et non des développements.

5.2 Poste client et agent d'inventaire

GLPI 10 intègre l'inventaire : aucune extension de type FusionInventory n'est requise, comme le rappelle le cahier de recettes. La mise en œuvre tient en trois points : activer l'inventaire côté serveur dans *Administration > Inventaire*, installer le GLPI Agent sur le poste Ubuntu client, puis le faire pointer sur l'URL du serveur.

Deux points d'attention propres à cette architecture. L'agent doit joindre le serveur à travers le tunnel, donc via son adresse sur **tun0** et non son adresse locale, sans quoi la règle **DOCKER-USER** bloquera ses

remontées. Et l'agent vérifie le certificat TLS : face à un certificat auto-signé, lui distribuer l'autorité de certification plutôt que désactiver la vérification, option toujours tentante et toujours mauvaise.

5.3 Collecte des courriels et création automatisée de tickets

Le cahier de recettes évoque un « *script* » et invite à regarder ceux qui existent déjà. La bonne réponse est qu'aucun script n'est nécessaire : GLPI dispose d'un **récepteur de courriels** natif, à configurer dans *Configuration > Récepteurs de courriels*. Il faut une boîte dédiée, un accès IMAPS en 993/tcp, et l'activation de l'action automatique **mailgate** – typiquement toutes les cinq minutes.

Côté réseau, la politique **default allow outgoing** déjà en place autorise cette sortie : aucune règle supplémentaire à prévoir. Côté exploitation, deux garde-fous méritent d'être posés d'entrée : un filtrage anti-spam en amont, faute de quoi chaque message indésirable crée un ticket, et une règle de rejet des messages automatiques pour éviter les boucles de notification.

5.4 Outil de contrôle à distance

Le cahier de recettes propose VNC ou TeamViewer. **Ces deux options ne sont pas équivalentes au regard de l'architecture retenue.** TeamViewer fonctionne en établissant une connexion sortante vers un relais hébergé chez l'éditeur : le poste devient joignable de l'extérieur par un canal qui échappe intégralement au pare-feu et au VPN, ce qui contredit directement la consigne « n'autoriser que le tunnel du VPN en entrée ».

La solution cohérente est un serveur VNC (**x11vnc** ou TightVNC) installé sur le poste client, écoutant **uniquement sur l'interface du tunnel**, le technicien s'y connectant après avoir monté le VPN. VNC ne chiffrant pas nativement, le tunnel fournit ici le chiffrement qui lui manque : l'architecture rend donc l'option la plus simple aussi la plus sûre. Un lien vers la session peut ensuite être ajouté dans la fiche de l'ordinateur côté GLPI.

5.5 Double authentification

Niveau	Mise en œuvre	Priorité
Application GLPI	TOTP native dans GLPI 10 (<i>Configuration > Authentification</i>), à imposer au minimum aux profils Super-Admin et Technicien	Haute
Accès SSH	libpam-google-authenticator , en conservant impérativement une session ouverte pendant la mise en place pour ne pas se verrouiller dehors	Moyenne
Tunnel VPN	Ajout d'un second facteur à l'authentification par certificat via auth-user-pass couplé à un module PAM TOTP	Moyenne

La priorité va sans hésitation à GLPI : c'est là que se trouvent les données, et c'est la seule des trois couches dont l'accès repose aujourd'hui sur un simple mot de passe.

5.6 Système de détection et de prévention d'intrusion Suricata

Le cahier de recettes signale d'emblée le point délicat : « *Attention aux fichiers de configuration* ». Deux décisions structurent le déploiement.

Mode de fonctionnement. En mode IDS, Suricata observe et alerte ; en mode IPS, il bloque, en s'insérant dans la chaîne de traitement des paquets via `NFQUEUE`. Le cahier de recettes demande un IPS, ce qui suppose de coordonner la règle `NFQUEUE` avec les règles `iptables` déjà posées – dont celle de la chaîne `DOCKER-USER` – sous peine de conflits d'ordre d'évaluation ou de coupure du service.

Points de configuration. Dans `suricata.yaml`, `HOME_NET` doit déclarer les deux réseaux réellement en jeu, `10.8.0.0/24` pour le tunnel et `192.168.1.0/24` pour le réseau local ; les interfaces à surveiller sont `tun0` et `enp0s8` ; le jeu de règles s'installe et se met à jour avec `suricata-update` (règles ET Open).

La limite à anticiper : le chiffrement aveugle la sonde

Le trafic GLPI est chiffré de bout en bout par NGINX, et le trafic VPN l'est aussi. Suricata ne verra donc **ni les requêtes HTTP, ni le contenu du tunnel** : les règles de détection applicative web seront inopérantes. Trois orientations restent pertinentes : surveiller ce qui demeure visible (balayages de ports, tentatives de connexion répétées sur 1194/udp et 22/tcp, anomalies TLS), placer la sonde en aval du proxy sur le réseau des conteneurs où le trafic redevient clair, ou activer la journalisation TLS pour détecter certificats et destinations suspects. Un IPS posé sans cette analyse préalable donnerait un sentiment de couverture largement illusoire.

6. Registre des points de vigilance

État des écarts connus sur l'existant à la date de rédaction, classés par priorité décroissante. Ce registre est destiné à être tenu à jour au fil des interventions.

#	Constat	Conséquence	Action	Réf.
1	La règle iptables d'isolation des conteneurs n'est pas persistante	L'isolation disparaît au redémarrage, sans signal visible : GLPI redevient joignable hors VPN	Unité systemd docker-isolation	3.6.1
2	Aucune sauvegarde n'est planifiée	La perte de la machine virtuelle entraîne celle du parc et de l'historique des tickets	Script et tâche planifiée	4.3
3	Les comptes tech , normal et post-only conservent leur mot de passe par défaut	Identifiants publiquement documentés, testés par les outils de balayage	Changer ou supprimer ces comptes	2.8, 3.7
4	Les données de démonstration sont toujours actives	5 400 ordinateurs et 1 500 tickets fictifs faussent tout indicateur	Désactiver depuis le tableau de bord	2.8
5	Le certificat TLS est auto-signé et sans subjectAltName	Avertissement permanent du navigateur, qui habitue les utilisateurs à passer outre les alertes	Certificat d'autorité interne, ou a minima régénérer avec -addext	3.1
6	Les certificats VPN sont valables 3 650 jours	Fenêtre de compromission de dix ans	Réémettre sur une validité d'un an	3.4.1
7	Les images utilisent l'étiquette latest	Un pull peut livrer une version majeure et migrer la base de manière irréversible	Épingler des versions explicites	4.6
8	Aucune double authentification n'est active	L'accès à l'ensemble des données du parc ne dépend que d'un mot de passe	Activer le TOTP natif de GLPI	5.5
9	L'échéance du certificat n'est pas surveillée	Interruption de service brutale à l'expiration	Tâche planifiée checkend	4.5
10	Le fichier .env a été affiché en clair lors du déploiement	Mots de passe de la base à considérer comme divulgués	Régénérer les mots de passe et exclure .env du versionnement	2.4

Annexe A. docker-compose.yml

Fichier de référence, tel que versionné dans le dépôt. Emplacement sur le serveur : `~/ticketing-projet/glpi/docker-compose.yml`.

```
version: '3.8'

services:
  db:
    image: mariadb:10.11
    container_name: glpi_db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  glpi:
    image: diouxx/glpi:latest
    container_name: glpi_web
    restart: always
    environment:
      - TIMEZONE=Europe/Paris
    depends_on:
      - db
    volumes:
      - glpi_data:/var/www/html/glpi

  nginx:
    image: nginx:latest
    container_name: glpi_proxy
    restart: always
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ../nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ../nginx/certs:/etc/nginx/certs:ro
    depends_on:
      - glpi

volumes:
  db_data:
  glpi_data:
```

L'attribut `version: '3.8'` de la première ligne est ignoré par Docker Compose v2 et provoque un avertissement à chaque commande. Il peut être supprimé sans conséquence.

Annexe B. nginx.conf

Emplacement sur le serveur : `~/ticketing-projet/nginx/nginx.conf`, monté dans le conteneur sur `/etc/nginx/conf.d/default.conf` en lecture seule.


```

server {
    listen 80;
    server_name _;
    # On force la redirection de HTTP vers HTTPS
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    # Emplacement des certificats qu'on vient de créer
    ssl_certificate /etc/nginx/certs/glpi.crt;
    ssl_certificate_key /etc/nginx/certs/glpi.key;

    # Paramètres de sécurité de base
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        # On renvoie le trafic vers le conteneur GLPI sur son port 80 interne
        proxy_pass http://glpi:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Annexe C. Jeu de règles réseau de référence

État attendu du filtrage après application de l'ensemble des procédures, correctif de persistance inclus. À comparer à la sortie réelle lors d'un contrôle.

```

# ---- UFW -----
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp          # administration SSH
sudo ufw allow 1194/udp        # entree du tunnel OpenVPN
sudo ufw allow in on tun0     # tout ce qui vient du tunnel
sudo ufw enable
# NB : 443/tcp n'est volontairement PAS ouvert.

# ---- iptables : isolation des conteneurs -----
sudo iptables -I DOCKER-USER -i enp0s8 -j DROP
# rendue persistante par l'unité systemd docker-isolation.service

# ---- controles -----
sudo ufw status verbose
sudo iptables -L DOCKER-USER -n -v --line-numbers
curl -Ik --max-time 5 https://192.168.1.17/ # hors tunnel : doit ECHOUER
curl -Ik --max-time 5 https://10.8.0.1/     # dans le tunnel : doit REPONDRE

```

Annexe D. Table des figures

Correspondance entre les figures de cette documentation et les fichiers de capture d'origine, disponibles dans le répertoire [capture/](#) du dépôt.

Figure	Fichier source	Objet
1	1-MiseAJourVM.PNG	Résultat attendu : la liste des paquets est actualisée et la mise à niveau démarre
2	2-InstallationDocker.PNG	Installation du moteur Docker et de Docker Compose
3	3-EtatDocker.PNG	Contrôle de la version

4	4-FichierEnvAvecMdpMYSQL.PNG	Structure du fichier .env telle que créée lors du déploiement initial
5	5-EcritureDockerCompsoe.PNG	Fichier de composition initial : MariaDB et GLPI
6	6-LancementDockerCompose.PNG	Résultat attendu : création du réseau et des volumes, téléchargement des images, puis démarrage des...
7	7-EtatImagesGLPI.PNG	Les deux conteneurs sont à l'état Up. glpi_db n'expose 3306 qu'en interne, tandis que glpi_web publie...
8	8-ConfigurationGLPI.PNG	Étape 6 : l'installation est terminée
9	9-PremiereConnexionGLPI.PNG	Première connexion
10	10-ModificaionMdpParDefaut.PNG	Changement du mot de passe du compte Super-Admin glpi
11	9-GenerationDuCertificatSSL.PNG (recadrée)	Génération du certificat auto-signé, RSA 2048 bits, valable 365 jours
12	11-ConfigurationNGINX.PNG	Configuration du reverse proxy
13	12-AjoutNGINXDansDockerCompose.PNG	Fichier de composition final
14	13-LancementAvecNGINX.PNG	Résultat attendu : quatre éléments démarrés – le réseau et les trois conteneurs
15	14-AccessGLPIEnHTTPS.PNG	GLPI répond en HTTPS
16	15-InstallationOpenVPN.PNG	Installation en mode non interactif
17	20-ConfigurationPareFeuServeur.PNG	Mise en place du jeu de règles
18	21-BlocageTraficVersDocker.PNG	Blocage de tout trafic à destination des conteneurs arrivant par l'interface physique